

INTERNSHIP REPORT ON COLLEGE MANAGEMENT SYSTEM USING PYTHON, TKINTER AND MYSQL

*(Submitted in partial fulfilment of the requirement for the Internship Course
Code: 24L6.0-CSC-INT, of B.Tech Computer Science Programme)*

By

Paras Yadav



Student Id: [Instituion No]

Department of Computer Science

Zillionsofttech Pvt.Ltd

Session: 2026–27

Year of Submission: 2026

CERTIFICATE

Certified that this Internship Report entitled “**College Management System using Python, Tkinter and MySQL**” is an original report of the work done by me under the guidance of Internship Mentor **Mr. Kamal Kumar**, Trainer, ZillionSofTech, Rewari, and under the supervision of Internship Supervisor **Ms. Savita** , Department of Computer Science, submitted as a part of the Internship Course of the Data Analytics Programme (B.Tech Computer Science) of ZillionSoftech Pvt .Ltd , Rewari

Dated: _____

Signature of Student: _____

Name: Paras Yadav

Student ID: [instution id]

Countersigned by:

Internship Supervisor

ZillionSoftech Pvt.Ltd

ACKNOWLEDGMENTS

The successful completion of this internship would not have been possible without the guidance, support, and encouragement of many people, and I take this opportunity to express my sincere gratitude to all of them.

First and foremost, I am deeply thankful to **Mrs Savtia , Department of Computer Science, ZillionSoftech Pvt . Ltd**, for granting me the permission and providing the academic framework to undertake this internship as a part of my postgraduate programme.

I express my heartfelt gratitude to **ZillionSofTech, Rewari** for giving me the opportunity to work as an intern and for providing a professional and supportive working environment throughout the internship period.

I am especially grateful to my Internship Mentor, **Mr. Kamal Kumar, Trainer**, whose constant guidance, technical expertise, and patient mentoring helped me understand real-world software development practices in Python and complete my assigned project successfully.

I sincerely thank my Internship Supervisor, **/Ms. Savita**, Department of Computer Science, for his/her valuable suggestions, regular monitoring, and academic guidance during the entire internship duration.

Finally, I would like to thank my parents, friends, and all the team members of the organization who directly or indirectly supported me during this internship.

Paras Yadav

Table of Contents

CERTIFICATE	2
ACKNOWLEDGMENTS	3
1. ABOUT THE ORGANIZATION.....	6
1.1 Vision and Mission	6
1.2 Areas of Work.....	6
1.3 Organizational Structure	6
1.4 Work Culture and Facilities	6
1.5 Why This Organization Was Chosen.....	6
2. OBJECTIVES OF THE INTERNSHIP	8
2.1 General Objectives.....	8
2.2 Technical Objectives.....	8
2.3 Expected Outcome	9
3. DETAILS OF WORK DONE	10
3.1 Project Overview	10
Main Features of the Application	10
3.2 Technologies and Tools Used.....	11
3.3 System Architecture.....	12
Flow of a Typical Operation (College Mangement System)	12
3.4 Project Directory Structure	13
3.5 Database Design.....	15
r3.5.1 Tables in the Database.....	15
3.5.2 Structure of the students Table	15
3.5.3 Structure of the fees Table	16
3.5.4 Relationships between Tables.....	16
3.6 Description of Modules.....	18
3.6.1 Login Module.....	18
3.6.2 Dashboard Module.....	18
3.6.3 Student Module.....	18
3.6.4 Course Module.....	18
3.6.5 Teacher Module	18
3.6.6 Fee Module	19

3.6.7 Department Module	19
3.6.8 Subject Module	19
3.6.9 Attendance Module	19
3.6.10 Enrollment Module	19
3.6.11 Reports Module.....	19
3.6.12 Utilities and Logging	20
3.7 Sample Code Snippets	21
<i>Snippet 1: MySQL Database Connection (database/db_connection.py)</i>	21
<code>import mysql.connector.....</code>	21
<i>Snippet 2: Insert Operation in DAO Layer (dao/student_dao.py)</i>	21
<i>Snippet 3: Add Student Form using Tkinter (views/student/add_student.py)</i>	22
<i>Snippet 4: Input Validation (services/validation_service.py)</i>	29
3.8 Testing of the Application	31
3.9 Week-wise Work Schedule.....	31
3.10 Roles and Responsibilities	32
3.11 Screenshots of the Application	33
4. LEARNING EXPERIENCES	37
4.1 Technical Experiences	37
4.2 Professional Experiences	37
4.3 Challenges Faced and How They Were Overcome	38
5. LEARNING OUTCOMES	39
6. CONCLUSION / SUMMARY	40
7. ATTACHMENTS.....	Error! Bookmark not defined.

1. ABOUT THE ORGANIZATION

ZillionSofTech, Rewari is a technical training and software development organization located at B9 Basement, Palika Complex ,Model Town, Rewari ,Haryana . Established in [2018], the organization works in the areas of software development, desktop and web application development, data analytics, and IT skill training. The organization conducts industry-oriented internship programmes in which students work on live, real-world projects under the mentorship of experienced professionals.

1.1 Vision and Mission

The vision of the organization is to bridge the gap between academic learning and industry requirements by producing skilled, employable, and industry-ready professionals. Its mission is to impart practical, hands-on training on widely used technologies such as Python, MySQL, Java, and data analytics tools, and to deliver quality software solutions to its clients.

1.2 Areas of Work

- Application development using Python (Desktop GUI applications using Tkinter, automation scripts, and data processing)
- Database design and management using MySQL
- Data Analytics using Python (Pandas, NumPy), Excel, and Power BI
- Web Development and Digital Marketing
- Corporate training, student internship, and placement assistance programmes

1.3 Organizational Structure

The organization functions under a Director, supported by project managers, senior developers, trainers, and support staff. Interns are attached to a project team and work under the direct guidance of a senior developer/trainer who acts as the Internship Mentor. Weekly review meetings are conducted to monitor the progress of the interns and the project deliverables.

1.4 Work Culture and Facilities

The work culture of the organization is collaborative and learning-oriented. Daily task allotment, regular code reviews, task tracking, and proper documentation are integral parts of the workflow. The organization provides a fully equipped computer lab with the required software (Python, MySQL Server, MySQL Workbench, VS Code/PyCharm) and internet facility, which helped me understand the professional practices followed in the software industry.

1.5 Why This Organization Was Chosen

I selected this organization for my internship because it offers project-based training on Python and MySQL, which are directly related to my area of study and career interest. Being a local organization, it was also convenient in terms of accessibility, and it fulfils the criteria of an Internship Providing Organization (IPO) as laid down in the Internship Guidelines of ZillionSoftech Pvt . Ltd .

2. OBJECTIVES OF THE INTERNSHIP

The internship was undertaken as a compulsory 4-credit course of 4–6 weeks duration (120 hours) as per the Curriculum and Credit Framework of ZillionSoft Pvt.Ltd , framed in accordance. The internship falls under the category of “Internship for Enhancing the Employability”. The main objectives of this internship were:

2.1 General Objectives

- To gain first-hand practical experience of working in a professional software development environment and to understand the working culture of an IT organization.
- To integrate the classroom/laboratory learning of the Data Analytics Computer Science programme with the actual workplace.
- To develop professional competencies such as teamwork, communication, time management, problem-solving, discipline, and dignity of labour.
- To enhance employability skills and build confidence for future job roles in the field of software development.

2.2 Technical Objectives

- To learn desktop application development in Python using the Tkinter GUI toolkit.
- To design and implement a relational database in MySQL and connect it with a Python application using the mysql-connector-python library.
- To understand and implement a layered software architecture (Models, DAO, Controllers, Views, Services, and Utilities) in a real project.
- To implement complete CRUD (Create, Read, Update, Delete) operations, user authentication, input validation, and report generation.
- To understand the complete Software Development Life Cycle (SDLC) including requirement analysis, design, development, testing, and documentation.
- To learn the use of industry-standard tools such as Git/GitHub for version control, VS Code/PyCharm for development, and MySQL Workbench for database management

2.3 Expected Outcome

At the end of the internship, the expected outcome was a fully working desktop-based College Management System developed in Python with a Tkinter GUI and a MySQL backend, along with complete project documentation, which could be used by a training institute to manage its student records, courses, teachers, and fee collection.

3. DETAILS OF WORK DONE

3.1 Project Overview

During the internship, I was assigned to work on a live project titled **“College Management System” – a desktop-based application developed using Python, Tkinter, and MySQL**. The objective of the project was to develop an application for a training institute to computerize its day-to-day activities such as managing student admissions, courses, teachers, fee collection, and report generation, which were earlier maintained manually in registers.

The application follows a layered architecture in which the GUI (Views), the business logic (Controllers and Services), and the database operations (DAO – Data Access Objects) are kept in separate layers. This separation of concerns makes the application easy to understand, test, maintain, and extend.

Main Features of the Application

- Secure Login System with username/password authentication for authorized users.
- Dashboard displaying quick access to all modules and overall management features.
- Student Management: Add, Update, Delete, Search, and View student records.
- Teacher Management: Add, Update, Delete, Search, and View teacher records.
- Department Management: Add, Update, Delete, Search, and Manage department details.
- Course Management: Add, Update, Delete, Search, and Manage course information.
- Subject Management: Add, Update, Delete, Search, and Manage subject details.
- Fee Management: Add, Update, Delete, Search, and Manage student fee records.
- Attendance Management: Record, Update, Delete, Search, and View student attendance.
- MySQL Database (college_db2) used for secure storage and management of all records.
- MVC (Model-View-Controller) Architecture for better code organization and maintainability
- User-friendly Tkinter GUI with forms, tables (Treeview), message boxes, and navigation.
- Input validation and error handling to ensure accurate and reliable data entry.

- Complete CRUD (Create, Read, Update, Delete) operations for all major modules.

3.2 Technologies and Tools Used

Category	Technology / Tool	Purpose in the Project
Programming Language	Python 3.12	Developing the core logic and backend functionality of the College Management System.
GUI Toolkit	Tkinter (with tk widgets)	Designing forms, windows, buttons, labels, Treeview tables, and user interface.
Database	MySQL 8.0	(college_db2) Storing and managing student, teacher, department, course, subject, fee, and attendance records.
DB Connector	mysql-connector-python	Connecting the Python application with the MySQL database and executing SQL queries.
Database Tool	MySQL Workbench	Creating, managing, and testing the college_db2 database and tables.
IDE / Editor	VS Code	Writing, debugging, and managing the project source code
Version Control	Git and GitHub	Managing project versions and maintaining the project repository
Architecture	MVC (Model–View–Controller)	Organizing the project into Model, View, Controller, and DAO layers for better maintainability.

3.3 System Architecture

The project follows a layered (MVC-like) architecture. Each layer has a single, well-defined responsibility, and the layers communicate with each other in a fixed direction, as described below:

- **Views (Tkinter GUI):** This layer contains all the screens of the application (login window, dashboard, and forms for students, courses, teachers, fees, subject, department, attendance and reports). The views only collect input from the user and display output; they do not contain any database code.
- **Controllers:** Controllers act as a bridge between the views and the DAO layer. When the user clicks a button (e.g., “Save Student”), the view calls the corresponding controller, which validates the data through the validation service and then calls the appropriate DAO method.
- **Services:** Common business logic such as authentication (auth_service.py), input validation (validation_service.py), report preparation (report_service.py), and file export (export_service.py) is written in the services layer so that it can be reused by multiple controllers.
- **DAO (Data Access Objects):** This layer contains all the SQL queries of the application. Each entity has its own DAO class (StudentDAO, CourseDAO, TeacherDAO, FeeDAO, DepartmentDAO , Attendance DAO, Enrollment DAO , Subject DAO) that performs INSERT, SELECT, UPDATE, and DELETE operations using parameterized queries to prevent SQL injection.
- **Models:** Model classes (Student, Course, Teacher, Fee, Attendance , Department , Enrollment) represent one record of the corresponding table as a Python object, which makes passing data between the layers clean and readable.
- **Database (MySQL):** The MySQL database college_db2 permanently stores all the data of the application in five related tables.

Flow of a Typical Operation (College Mangement System)

User logs into the College Management System → the Dashboard opens successfully → the user selects a module such as Student, Teacher, Department, Course, Subject, Fee, or Attendance → enters or updates the required information → the corresponding Controller receives the request → the DAO layer performs the required database operation on the college_db2 database → the result is returned to the View → a success or error message is displayed, and the Treeview is refreshed to show the latest records.

3.4 Project Directory Structure

The complete project was organized in the following directory structure, which reflects the layered architecture described above:

```
StudentManagementSystem/
|-- main.py                # Application entry point
|-- requirements.txt       # Required Python libraries
|-- README.md             # Project documentation
|
|--Config
|   |-- db.py
|
|-- database/              # Database connection & creation scripts
|   |-- db_connection.py   # MySQL database connection
|   |-- create_database.py # Creates database & tables
|   |-- backup.py          # Database backup utility
|   |-- sql/
|       |--college_db2.sql # SQL script of the schema
|
|-- models/                # Data model classes
|   |-- student.py
|   |--course.py
|   |--teacher.py
|   |--fee.py
|   |--subject.py
|   |--attendance.py
|   |--enrollment.py
|   |--user.py
|
|-- dao/                   # Data Access Object layer (SQL operations)
|   |-- student_dao.py, course_dao.py, teacher_dao.py,
|   |-- fee_dao.py, user_dao.py, database_dao.py, subject_dao.py,
|   |--department_dao.py, attendance_dao.py, enrolment_dao.py
|
|-- controllers/           # Connects views with DAO layer
|   |-- subject_controller.py, student_controller.py,
|   |-- course_controller.py, teacher_controller.py,
|   |-- fee_controller.py, department_controller.py,
|   |--attendance_controller.py, enrollment_controller.py
|   |--login_controller.py
|
|-- views/                 # Tkinter GUI screens
|   |-- login.py, dashboard_view.py, course_view.py, subject_view.py,
|   |-- student_view.py, teacher_view.py, department_view.py, fee_view.py,
|   |--attendance_view.py, enrollment_vire.py, reports_view.py
|
|-- services/              # Business logic services
|   |--validation_service.py, export_view.py,
```

```
| |-- report_service.py

|
|-- utils/                # Helper utilities
| |-- constants.py, helper.py, validator.py,
| |--messagebox_util.py
|
|-- assets/
| |-- icons/
| |--image/
|
|                # Images, icons, logo
|-- reports/
|
|                # Generated PDF / Excel / CSV reports
|-- logs/
|
|                # application.log
|-- tests/
| |--student_test.py
| |--teacher_test.py
| |--course_test.py
| |--subject_test.py
| |--login_test.py                # Unit tests (student, course, login)
```

3.5 Database Design

A relational database named **college_db3** was designed in MySQL after analyzing the requirements of the College Management System. The database is normalized up to the **Third Normal Form (3NF)** to reduce redundancy and improve data consistency. The database consists of the following tables.

3.5.1 Tables in the Database

Sr. No.	Table Name	Description
1	users	Stores login credentials of administrator and staff users.
2	students	Stores student personal and academic details.
3	courses	Stores course information offered by the college
4	teachers	Stores teacher details, qualifications and salary information.
5	departments	Stores department information such as department name and HOD.
6	subjects	Stores subject details and links them with courses.
7	enrollment	Stores enrollment records of students in courses.
8	attendance	Stores fee payment details, remaining amount and payment status.
9	fees	Stores fee payment details, remaining amount and payment status.

3.5.2 Structure of the students Table

Field Name	Data Type	Description / Constraint
student_id	INT	Primary Key, Auto Increment
First_name	VARCHAR(100)	Full name of the student, NOT NULL
last_name	VARCHAR(100)	last name of the student
dob	DATE	Date of birth
gender	ENUM('M', 'F', 'O')	Gender of the student
phone	VARCHAR(10)	10-digit mobile number, UNIQUE
email	VARCHAR(100)	Email address of the student
address	VARCHAR(255)	Residential address
course_id	INT	Foreign Key referencing courses(course_id)
Department_id	INT	Foreign Key → departments.department_id

3.5.3 Structure of the fees Table

Field Name	Data Type	Description / Constraint
fee_id	INT	Primary Key, Auto Increment
student_id	INT	Foreign Key referencing students(student_id)
Total_fee	DECIMAL(10,2)	Total Fee
Paid_fee	Decimal(10,2)	Paid Fee
remaining_fee	Decimal(10,2)	Remaining_Fee
status	Vaechar(20)	Paid / pending
Payment_date	Date	Payment Date

3.5.4 Relationships between Tables

□ **Departments** (1) ————— (Many) **Students:**
One department can have many students. The students.department_id field is a foreign key referencing departments.department_id.

□ **Departments** (1) ————— (Many) **Teachers:**
One department can have many teachers. The teachers.department_id field is a foreign key referencing departments.department_id.

□ **Courses** (1) ————— (Many) **Subjects:**
One course can contain many subjects. The subjects.course_id field is a foreign key referencing courses.course_id.

□ **Students** (1) ————— (Many) **Fees:**
One student can have multiple fee payment records. The fees.student_id field is a foreign key referencing students.student_id.

□ **Students** (1) ————— (Many) **Attendance:**
One student can have many attendance records. The attendance.student_id field is a foreign key referencing students.student_id.

□ **Students** (Many) ————— (Many) **Courses:**
Students can enroll in multiple courses through the **Enrollment** table. The enrollment.student_id and enrollment.course_id fields are foreign keys referencing the students and courses tables respectively.

The fee management module calculates the **remaining fee** using the following formula:

Remaining Fee = Total Fee – Paid Fee

This ensures that the fee status remains accurate and consistent within the system.

3.6 Description of Modules

3.6.1 Login Module

The login module is the entry point of the application (views/login.py). The user enters a username and password, which are verified by auth_service.py against the users table. Passwords are never stored in plain text; they are hashed using the SHA-256 algorithm (hashlib) before storing and comparing. On successful login, the dashboard opens according to the role of the user; on failure, an error message box is displayed and the attempt is logged.

3.6.2 Dashboard Module

The dashboard (views/dashboard.py) is the main window of the application. It displays summary cards showing the total number of active students, courses, teachers, and the total fee collected in the current month. It also contains a menu/side panel through which the user can navigate to all the other modules of the application.

3.6.3 Student Module

This is the core module of the project. It provides five screens — Add Student, Update Student, Delete Student, Search Student, and Student List. The student list is displayed in a ttk.Treeview widget with scrollbars, and a student can be searched by name, mobile number, or course. All inputs (mobile number, email, date of birth) are validated before saving.

3.6.4 Course Module

The course module manages the courses offered by the institute, such as Data Analytics, Full Stack Development, and Digital Marketing. Each course has a name, duration (in months), and total fee. A course that has students enrolled in it cannot be deleted, which maintains referential integrity.

3.6.5 Teacher Module

The teacher module stores the details of teachers/trainers along with their qualification and the subjects assigned to them. The Add Teacher and Update Teacher screens work in the same pattern as the student module.

3.6.6 Fee Module

The fee module contains three screens: Collect Fee (records a new payment with amount, date, and mode), Fee History (shows all payments of a selected student), and Pending Fee (lists all students whose paid amount is less than the course fee, using a JOIN and GROUP BY query). A fee receipt can be printed/exported after each collection.

3.6.7 Department Module

The Department Module maintains department information such as department name and Head of Department (HOD). It allows users to add, update, delete, search, and display department records stored in the **departments** table.

3.6.8 Subject Module

The Subject Module manages subject details for each course. It allows users to add, update, delete, search, and display subject information such as subject name, semester, and course. The records are stored in the **subjects** table.

3.6.9 Attendance Module

The Attendance Module records daily student attendance. It stores attendance date, student ID, and attendance status (Present or Absent). Users can add, update, delete, search, and display attendance records.

3.6.10 Enrollment Module

The Enrollment Module manages student enrollment into courses. It stores student ID, course ID, enrollment date, and enrollment status. Users can add, update, delete, search, and display enrollment records.

3.6.11 Reports Module

The reports module generates the Student Report, Fee Report, and Attendance Report. Using `report_service.py` and `export_service.py`, every report can be exported to PDF (reportlab), Excel (openpyxl), or CSV format and is saved in the `reports/` directory with a date-stamped file name.

3.6.12 Utilities and Logging

The `utils/` package contains common helpers — constants, validators (mobile, email, date), date utilities, and a message box wrapper for showing uniform success/error/confirmation dialogs. All important events and errors of the application are written to `logs/application.log` using Python's logging module, which was very useful during debugging.

3.7 Sample Code Snippets

Snippet 1: MySQL Database Connection (database/db_connection.py)

```
import mysql.connector

class Database:

    @staticmethod
    def get_connection():

        return mysql.connector.connect(

            host="localhost",

            user="root",

            password="your_password",

            database="college_db3"
```

Snippet 2: Insert Operation in DAO Layer (dao/student_dao.py)

```
def save(self, student):

    conn = Database.get_connection()
    cursor = conn.cursor()

    query = """
    INSERT INTO students
    (
        first_name,
        last_name,
        gender,
        dob,
        phone,
        email,
        address,
        department_id
    )
    VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
    """

    cursor.execute(query, student.to_tuple())

    conn.commit()

    cursor.close()
    conn.close()
```

Snippet 3: Add Student Form using Tkinter (views/student/add_student.py)

```
import tkinter as tk
from tkinter import ttk, messagebox

from controller.student_controller import StudentController

class StudentView:

    def __init__(self):

        self.controller = StudentController()

        self.root = tk.Toplevel()

        self.root.title("Student Management")
        self.root.geometry("1100x650")
        self.root.resizable(True, True)

        self.create_widgets()

    def create_widgets(self):

        title = tk.Label(
            self.root,
            text="Student Management",
            font=("Arial", 20, "bold")
        )
        title.pack(pady=10)

        form = tk.Frame(self.root)
        form.pack(pady=10)

        # =====
        # Row 0
        # =====

        tk.Label(form, text="Student ID").grid(row=0, column=0, padx=10,
pady=5)

        self.student_id = tk.Entry(form, width=25)
        self.student_id.grid(row=0, column=1)

        tk.Label(form, text="First Name").grid(row=0, column=2, padx=10)

        self.first_name = tk.Entry(form, width=25)
        self.first_name.grid(row=0, column=3)

        # =====
        # Row 1
```

```

# =====

tk.Label(form, text="Last Name").grid(row=1, column=0, padx=10,
pady=5)

self.last_name = tk.Entry(form, width=25)
self.last_name.grid(row=1, column=1)

tk.Label(form, text="Gender").grid(row=1, column=2)

self.gender = ttk.Combobox(
    form,
    values=["Male", "Female", "Other"],
    width=22,
    state="readonly"
)

self.gender.grid(row=1, column=3)

# =====
# Row 2
# =====

tk.Label(form, text="Date of Birth").grid(row=2, column=0)

self.dob = tk.Entry(form, width=25)
self.dob.grid(row=2, column=1)

tk.Label(form, text="Phone").grid(row=2, column=2)

self.phone = tk.Entry(form, width=25)
self.phone.grid(row=2, column=3)

# =====
# Row 3
# =====

tk.Label(form, text="Email").grid(row=3, column=0)

self.email = tk.Entry(form, width=25)
self.email.grid(row=3, column=1)

tk.Label(form, text="Address").grid(row=3, column=2)

self.address = tk.Entry(form, width=25)
self.address.grid(row=3, column=3)

# =====
# Row 4
# =====

```

```

tk.Label(form, text="Department ID").grid(row=4, column=0)

self.department_id = tk.Entry(form, width=25)
self.department_id.grid(row=4, column=1)

# =====
# Buttons
# =====

button_frame = tk.Frame(self.root)
button_frame.pack(pady=15)

tk.Button(
    button_frame,
    text="Save",
    width=15,
    command=self.save_student
).grid(row=0, column=0, padx=5)

tk.Button(
    button_frame,
    text="Update",
    width=15,
    command=self.update_student
).grid(row=0, column=1, padx=5)

tk.Button(
    button_frame,
    text="Delete",
    width=15,
    command=self.delete_student
).grid(row=0, column=2, padx=5)

tk.Button(
    button_frame,
    text="Search",
    width=15,
    command=self.search_student
).grid(row=0, column=3, padx=5)

tk.Button(
    button_frame,
    text="Show All",
    width=15,
    command=self.show_students
).grid(row=0, column=4, padx=5)

tk.Button(
    button_frame,
    text="Clear",
    width=15,

```



```

        command=self.clear_fields
    ).grid(row=0, column=5, padx=5)

# =====
# TreeView
# =====

columns = (
    "ID",
    "First Name",
    "Last Name",
    "Gender",
    "DOB",
    "Phone",
    "Email",
    "Address",
    "Department"
)

self.tree = ttk.Treeview(
    self.root,
    columns=columns,
    show="headings",
    height=15
)

for col in columns:
    self.tree.heading(col, text=col)
    self.tree.column(col, width=120)

self.tree.pack(fill="both", expand=True, padx=10, pady=10)

self.tree.bind(
    "<<TreeviewSelect>>",
    self.select_record
)

self.show_students()

# =====
# Save Student
# =====

def save_student(self):

    try:

```

```

        if self.first_name.get() == "":
            messagebox.showerror("Error", "First Name is Required")
            return

        self.controller.save_student(
            self.first_name.get(),
            self.last_name.get(),
            self.gender.get(),
            self.dob.get(),
            self.phone.get(),
            self.email.get(),
            self.address.get(),
            self.department_id.get()
        )

        messagebox.showinfo("Success", "Student Saved Successfully")

        self.clear_fields()
        self.show_students()

    except Exception as e:
        messagebox.showerror("Error", str(e))

# =====
# Update Student
# =====

def update_student(self):

    try:

        self.controller.update_student(
            self.student_id.get(),
            self.first_name.get(),
            self.last_name.get(),
            self.gender.get(),
            self.dob.get(),
            self.phone.get(),
            self.email.get(),
            self.address.get(),
            self.department_id.get()
        )

        messagebox.showinfo("Success", "Student Updated Successfully")

        self.clear_fields()
        self.show_students()

    except Exception as e:
        messagebox.showerror("Error", str(e))

```

```

# =====
# Delete Student
# =====

def delete_student(self):

    try:

        if self.student_id.get() == "":
            messagebox.showerror("Error", "Select Student")
            return

        self.controller.delete_student(
            self.student_id.get()
        )

        messagebox.showinfo("Success", "Student Deleted Successfully")

        self.clear_fields()
        self.show_students()

    except Exception as e:
        messagebox.showerror("Error", str(e))

# =====
# Search Student
# =====

def search_student(self):

    try:

        if self.student_id.get() == "":
            messagebox.showerror("Error", "Enter Student ID")
            return

        row = self.controller.search_student(
            self.student_id.get()
        )

        if row:

            self.clear_fields()

            self.student_id.insert(0, row[0])
            self.first_name.insert(0, row[1])
            self.last_name.insert(0, row[2])

            self.gender.set(row[3])

            self.dob.insert(0, row[4])

```

```

        self.phone.insert(0, row[5])
        self.email.insert(0, row[6])
        self.address.insert(0, row[7])
        self.department_id.insert(0, row[8])

    else:

        messagebox.showinfo(
            "Not Found",
            "Student Not Found"
        )

    except Exception as e:

        messagebox.showerror(
            "Error",
            str(e)
        )

# =====
# Show Students
# =====

def show_students(self):

    for item in self.tree.get_children():
        self.tree.delete(item)

    rows = self.controller.get_all_students()

    for row in rows:
        self.tree.insert("", tk.END, values=row)

# =====
# TreeView Selection
# =====

def select_record(self, event):

    selected = self.tree.focus()

    if selected == "":
        return

    values = self.tree.item(selected, "values")

    self.clear_fields()

    self.student_id.insert(0, values[0])
    self.first_name.insert(0, values[1])
    self.last_name.insert(0, values[2])

```

```

        self.gender.set(values[3])

        self.dob.insert(0, values[4])
        self.phone.insert(0, values[5])
        self.email.insert(0, values[6])
        self.address.insert(0, values[7])
        self.department_id.insert(0, values[8])

# =====
# Clear Fields
# =====

def clear_fields(self):

    self.student_id.delete(0, tk.END)
    self.first_name.delete(0, tk.END)
    self.last_name.delete(0, tk.END)

    self.gender.set("")

    self.dob.delete(0, tk.END)
    self.phone.delete(0, tk.END)
    self.email.delete(0, tk.END)
    self.address.delete(0, tk.END)
    self.department_id.delete(0, tk.END)

    show_error(result["message"])

```

Snippet 4: Input Validation (services/validation_service.py)

```

import re

class Validation:

    # =====
    # Empty Validation
    # =====

    @staticmethod
    def is_empty(value):

        if str(value).strip() == "":
            return True

        return False

    # =====
    # Phone Validation
    # =====

    @staticmethod
    def valid_phone(phone):

```

```

        pattern = r"^[6-9][0-9]{9}$"

        return re.match(pattern, phone)

# =====
# Email Validation
# =====

@staticmethod
def valid_email(email):

    pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"

    return re.match(pattern, email)

# =====
# Number Validation
# =====

@staticmethod
def is_number(value):

    try:

        float(value)

        return True

    except:

        return False

# =====
# Positive Number
# =====

@staticmethod
def is_positive(value):

    try:

        return float(value) >= 0

    except:

        return False

# =====
# Date Validation
# YYYY-MM-DD
# =====

@staticmethod
def valid_date(date):

    pattern = r"^\d{4}-\d{2}-\d{2}$"

    return re.match(pattern, date)

```

3.8 Testing of the Application

The College Management System was tested thoroughly after the implementation of all modules. Testing was performed to ensure that each module functions correctly, validates user input, and maintains data consistency within the MySQL database.

Unit Testing

The DAO classes, controllers, and model classes were tested individually. CRUD operations for Students, Teachers, Departments, Courses, Subjects, Fees, Attendance, and Enrollment were verified using valid and invalid inputs. Database connectivity and SQL queries were also tested to ensure correct data insertion, updating, searching, and deletion.

Manual / Functional Testing

Each screen of the application was manually tested to verify navigation, data validation, user interaction, and database operations. All forms were checked for proper error handling, successful CRUD operations, and correct display of records in the Treeview widget.

Relationships between tables were also verified using foreign key constraints.

After successful testing, the complete application was executed and all modules worked as expected without any major runtime errors.

3.9 Week-wise Work Schedule

Week	Module / Activity	Description of Work Done
Week 1	Orientation, Python & Tkinter Basics	Induction session and introduction to the team and project; revision of Python fundamentals (OOP, functions, exception handling); learned Tkinter widgets (Label, Entry, Button, Treeview, Frame) and geometry managers; set up the development environment (Python, MySQL, VS Code, Git).
Week 2	Database Design & Connectivity	Analysed the requirements of the College Management System; designed the ER diagram and normalized database schema; created the college_db2 database and all tables using SQL scripts; implemented db_connection.py using mysql-connector-python with parameterized queries.
Week 3	Login & Student Module	Developed the login window with password hashing (hashlib) and role-based access; developed the complete Student module — add, update, delete, search, and list students using Tkinter forms and Treeview; implemented DAO classes for all database operations.
Week 4	Course, Teacher & Fee Modules	Developed the Course and Teacher modules with full CRUD operations; developed the Fee module — fee collection form, fee history, and pending fee calculation using SQL JOIN queries; implemented the validation service for all input forms.
Week 5	Dashboard, Reports & Integration	Designed the main dashboard showing summary counts of students, courses, teachers, and fees collected; developed report generation and export to PDF (reportlab), Excel (openpyxl), and CSV; integrated all modules through controllers and tested the navigation flow.

3.10 Roles and Responsibilities

- ❑ Designed and implemented the complete **College Management System** using **Python, Tkinter, and MySQL**.
- ❑ Developed the application using the **MVC (Model–View–Controller)** architecture.
- ❑ Created normalized database tables and maintained relationships using foreign keys.
- ❑ Implemented complete CRUD operations for Student, Teacher, Department, Course, Subject, Fee, Attendance, and Enrollment modules.
- ❑ Developed DAO classes using parameterized SQL queries for secure database operations.
- ❑ Integrated the Tkinter GUI with the MySQL database.
- ❑ Implemented input validation and exception handling throughout the application.
- ❑ Performed unit testing and manual testing for all modules.
- ❑ Fixed runtime errors and improved application performance during development.
- ❑ Prepared project documentation, database design, and user manual.

3.11 Screenshots of the Application

The screenshots of the important screens of the developed application are given below:

Fig. 1: Login Window

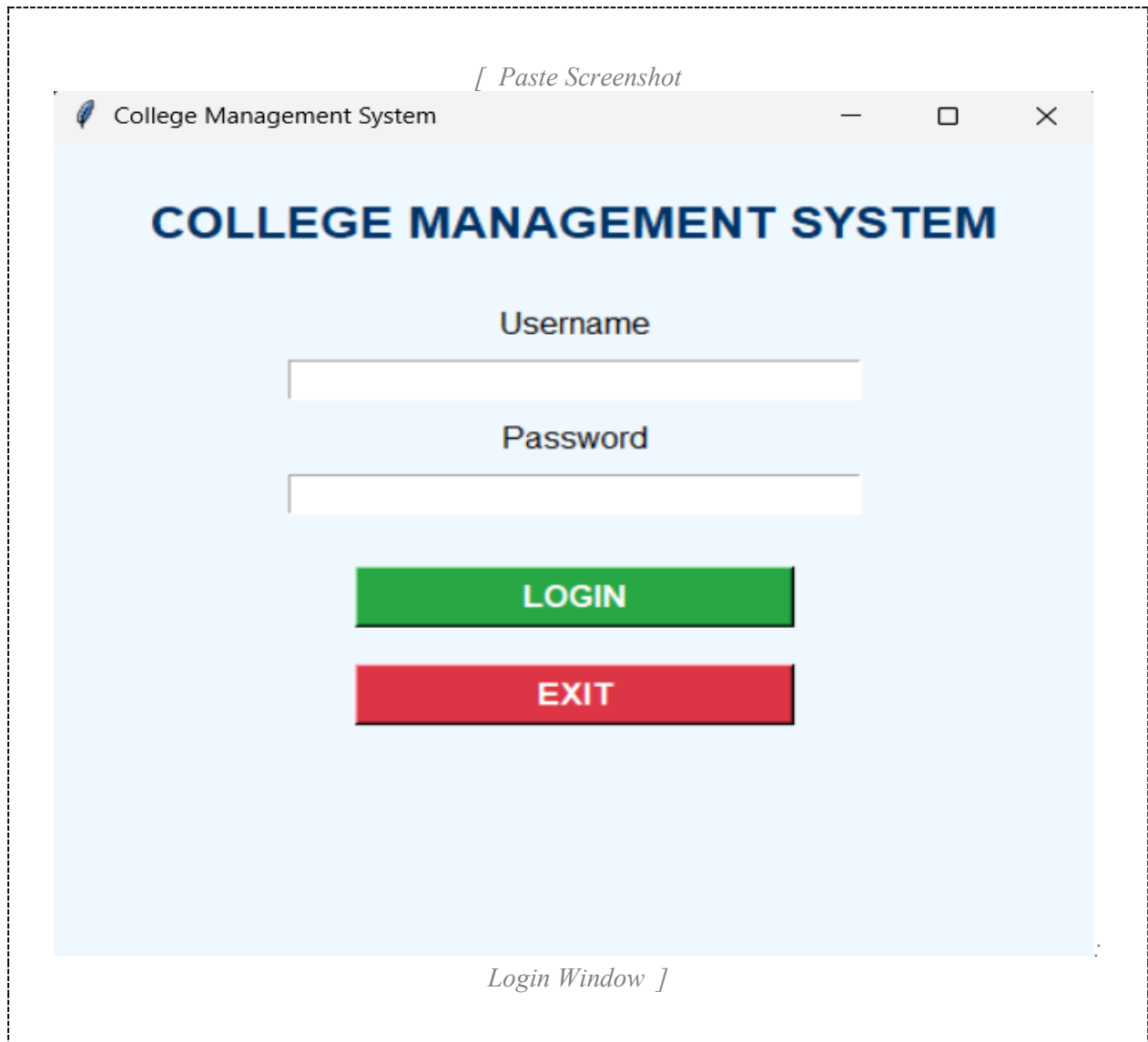


Fig. 2: Main Dashboard

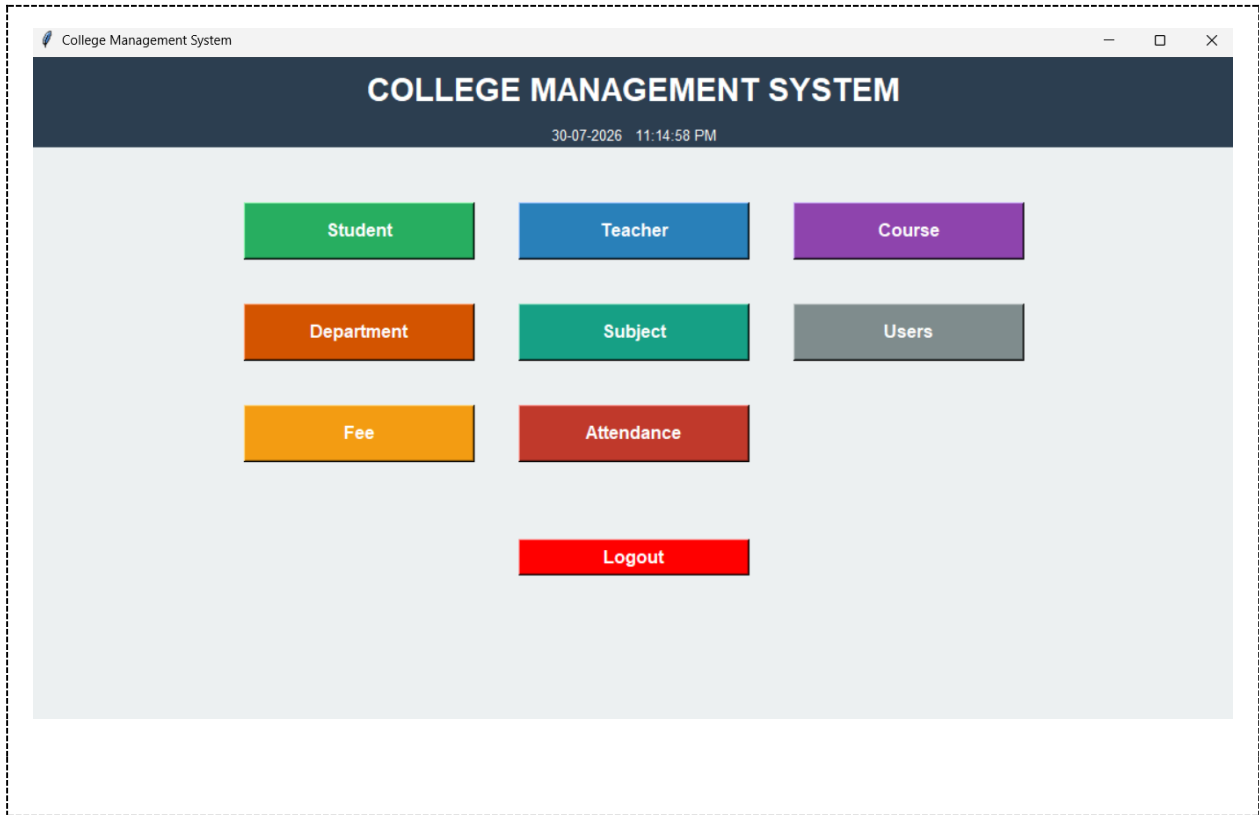


Fig. 3: Add Student Form

[Paste Scr]

Student Management

Student Management

Student ID

First Name

Last Name

Gender

Date of Birth

Phone

Email

Address

Department ID

Save

Update

Delete

Search

Show All

Clear

ID	First Name	Last Name	Gender	DOB	Phone	Email	Address	Department
1	Paras	Yadav	Male	2006-11-17	8855776699	paras567ydv@gmail.c	gurugram	1
4	Rahul	Sharma	Male	2005-06-11	9876543211	Rahul56776@gmail.co	Rewari	3
6	Ravi	Kumar	Male	2005-08-11	9988554745	Ravikum5676@gmail.c	Dharuehra	4
7	Rohan	Yadav	Male	2006-08-23	9998887777	Rohan56776@gmail.co	Delhi	5
8	Mayank	Kumar	Male	2004-11-23	8965742322	Maynak5676@gmail.co	Jaipur	6
9	Nihal	Tiwari	Male	2007-09-24	9632456987	Nihal2343@gmail.co	Gurugram	7

Fig. 4: Student List (Treeview)

ID	First Name	Last Name	Gender	DOB	Phone	Email	Address	Department
1	Paras	Yadav	Male	2006-11-17	8855776699	paras567ydv@gmail.co	gurugram	1
4	Rahul	Sharma	Male	2005-06-11	9876543211	Rahul56776@gmail.co	Rewari	3
6	Ravi	Kumar	Male	2005-08-11	9988554745	Ravikum5676@gmail	Dharuehra	4
7	Rohan	Yadav	Male	2006-08-23	9998887777	Rohan56776@gmail.co	Delhi	5
8	Mayank	Kumar	Male	2004-11-23	8965742322	Maynak5676@gmail.co	Jaipur	6
9	Nihal	Tiwari	Male	2007-09-24	9632456987	Nihal2343@gmail.co	Gurugram	7

Fig. 5: Teacher Main Screen

Teacher Management

Teacher Management

Teacher ID

First Name

Last Name

Gender

Phone

Email

Qualification

Salary

Department ID

Save

Update

Delete

Search

Show All

Clear

ID	First Name	Last Name	Gender	Phone	Email	Qualification	
1	Anju	Sharma	Female	9812254755	anju@34567gmail.com	Phd(Physics)	88000
2	Arun	kumar	Male	9992447551	arun5654@gmail.com	Phd(chemistry)	99000

4. LEARNING EXPERIENCES

The internship provided me with a rich and multi-dimensional learning experience that went far beyond classroom learning. Some of the key experiences are described below:

4.1 Technical Experiences

- Working on a live project helped me understand how a complete desktop application is planned, divided into layers and modules, developed, and integrated step by step, instead of writing everything in a single file as is often done in classroom practice.
- I understood the real importance of a layered architecture — when a bug appeared in the pending fee calculation, I only had to check the DAO and service layers without touching any GUI code.
- I learned professional database practices: normalization, foreign keys, JOIN and GROUP BY queries, parameterized queries to prevent SQL injection, and taking regular database backups.
- Building GUIs in Tkinter taught me practical concepts such as geometry management (grid/pack), event-driven programming, Toplevel windows, and displaying tabular data in Treeview widgets with scrollbars.
- I experienced the practical use of version control (Git) — committing daily work, writing meaningful commit messages, and restoring a previous version when a change broke the application.
- Debugging real errors using the application log file, reading official documentation, and searching for solutions independently improved my problem-solving ability significantly.

4.2 Professional Experiences

- Following a fixed daily schedule, meeting deadlines, and reporting work honestly in daily meetings taught me discipline and accountability.
- Participating in code reviews improved my communication skills and my ability to accept and implement constructive feedback.
- Preparing the user manual and demonstrating the project to non-technical staff taught me how to explain technical concepts in simple language.

- I observed how professionalism, punctuality, teamwork, and dignity of labour are valued in a real workplace.
-

4.3 Challenges Faced and How They Were Overcome

- **Understanding the existing structure:** In the first week, the multi-folder project structure was confusing. My mentor explained the flow of one complete operation (Add Student) across all layers, after which the entire structure became clear.
- **Database connectivity errors:** Initially I faced authentication and 'Access denied' errors while connecting Python to MySQL. Reading the error messages carefully and configuring the credentials in config.py resolved the issue.
- **Refreshing the Treeview:** After add/update/delete operations, the student list did not refresh automatically. I learned to clear and reload the Treeview after every successful database operation.
- **Pending fee logic:** Students having no fee record were not appearing in the pending list. Using a LEFT JOIN with IFNULL(SUM(amount_paid), 0) solved this problem and taught me an important SQL concept.

5. LEARNING OUTCOMES

On successful completion of the internship, the following learning outcomes were achieved:

- Ability to design and develop a complete desktop application in Python using the Tkinter GUI toolkit.
- Ability to design a normalized relational database in MySQL and connect it to a Python application using mysql-connector-python.
- Practical understanding of layered software architecture (Models – DAO – Controllers – Views – Services) and its benefits in real projects.
- Proficiency in implementing complete CRUD operations, JOIN-based reporting queries, and secure login with password hashing.
- Hands-on skill in exporting reports to PDF, Excel, and CSV formats using reportlab, openpyxl, and the csv module.
- Practical understanding of the Software Development Life Cycle (SDLC) — requirement analysis, design, coding, testing, and documentation.
- Competence in using professional tools: Git/GitHub, VS Code/PyCharm, MySQL Workbench, and Python's unittest and logging modules.
- Improved soft skills including teamwork, communication, time management, documentation, and presentation skills.
- Clear understanding of workplace culture, professional ethics, and the expectations of the IT industry, which has significantly enhanced my employability.

6. CONCLUSION / SUMMARY

The 4–6 week internship at **Zillions of Tech, Rewari** proved to be an extremely valuable component of my M.Sc. Computer Science programme. It gave me the opportunity to step out of the classroom and experience the actual working environment of the software industry.

During the internship, I successfully developed a complete College Management System using Python, Tkinter, and MySQL, following a professional layered architecture. I designed a normalized database, implemented secure login, full CRUD operations for students, courses, teachers, and fees, dynamic pending-fee calculation, and multi-format report generation. The application was unit tested, manually tested, documented, and demonstrated successfully.

The experience strengthened my technical foundation in Python programming and database management, sharpened my problem-solving and debugging skills, and developed professional qualities such as discipline, teamwork, and effective communication. I also understood how a real project moves through the phases of the Software Development Life Cycle, from requirement analysis to final delivery.

The internship has fulfilled its objectives of integrating classroom learning with the workplace, and the Internship Guidelines of ZillionSoft Pvt . Ltd It has given me clarity about my career direction and the confidence to take up job roles in software development and database management. I strongly believe that the knowledge and experience gained during this internship will serve as a strong foundation for my professional career.

The following documents are attached with this report:

- Internship Completion Certificate issued by the Internship Mentor (Appendix-IV).
- Consent of the Internship Mentor (Appendix-III).
- Additional screenshots of the developed application.
- SQL script of the database (college_db2.sql).
- Sample exported reports (PDF/Excel) generated by the application.
- Weekly attendance / work diary duly signed by the Internship Mentor.

[Attach the above documents after this page]